CSC3150 Lecture1 OS Basics

Basics

Three Hats of OS

Referee: 调控多应用并行

资源共享：现代 OS 可以强制调度 Preemptive Scheduling，即应用不能无限霸占 CPU。

90年代的 MacOS 9x 和 Windows 3.1及以前的 OS 采用 cooperative scheduling，就会是：A A A A A …

Illusionist: 抽象

Glue: 向应用提供公共服务

Evaluation Criteria of OS

Fairness (Equitable): 设计合理的资源分配策略

Virtual Machine

Type I Hypervisor (Native)

Type II Hypervisor (Hosted)

Lecture2 Fundamental Abstractions

Thread 线程：各自独占 CPU 的幻觉

OS 创造虚拟 CPU 并快速切换上下文给每个线程创造独占 CPU 的幻觉 (multiplex in time 时间复用)，因此线程是 CPU virtualization 的基本执行单位。

Address Space 内存地址：有序且庞大的幻觉

32 位处理器可以表示 232 个地址，约 4x109；64位则有 264 (~1.8x1019)，地址从 0 到 0xFFF...F。下图借用 CSC3060 课件展示虚拟地址分配：

Code Segment 和 Static Data Segment 都从 exe 文件加载，前者是机器指令 (PC 指向的 IMEM)，后者是 global variables 和 static variables (非临时变量)。

Stack 主要负责函数执行状态，比如方法 (包括 main()) 中的临时变量。每个线程都有自己独立的 Stack，用于 function calls、local variables、ra。

Heap 是用 malloc() 动态内存分配区域。 如果让一个指针 (临时变量) 执行分配的区域，指针本身还是存在栈上。编程时需要手动清理 Heap。

回忆 CSC3200/3060 的 * & 地狱：

(dt*) 表示强转换为 dt 数据类型；

dt * x/dt* x/dt *x 表示声明 dt 类指针，const dt* x 常量指针、dt * const x 指向常量的指针、const dt* const x 指向常量的常量指针

int *p = &x; 声明指针 p 指向 x 取地址后的结果

古代神秘召唤仪式：

```
const int (* const f)(const int *) = &function
// 初始化一个指向一个接受“指向整常量的指针”作为参数并输出整常量的函数function的常量指针f……
const int *f(const int *) {...}
// 声明一个接受“指向整常量的指针”作为参数并输出“指向整常量的指针”的函数f……
```

Process 进程：各自独占机器的幻觉

Process 是一个程序的受限权限的运行环境，= (Some) Address Space + Threads (≥1)*

进程还拥有虚拟内存、file descriptors、file system context、sockets 等的访问权

进程给每个程序划定一个边界，防止互相干扰或侵入 OS Kernel：

Fairness：OS 控制每个 process 能拿到多少 CPU time、Mem、I/O、disc…

Security and Privacy：恶意程序不能读写其他进程

Reliability：bug 无法跨进程造成内存损坏

Primary mechanism：限制虚拟空间到物理内存的映射 (translation)，线程和进程都只能碰进程的虚拟内存空间

Threads encapsulate concurrency，Address spaces encapsulate protection.

进程提供 address space，线程提供 execution flow。

Multiprogramming; Multithreading 多线程

单进程多线程 (Multithreading)：共享同一进程的虚拟地址空间，独立保存执行状态 (PC + Registers + SP + Stack)，以实现——

并行 (parallelism)：实现多核处理器等硬件并行

并发 (concurrency)：OS Context Switch (Virtual CPU) 交错推进

OS 控制 CPU 保存旧线程 PC，SP，regs 到内存，载入新线程 PC，SP，regs

切换原因：Timer (Preemptive Scheduling)、Voluntary yield (cooperative scheduling)、I/O (比如读写磁盘等超慢指令出发切换)……

如果没有 Process 呢?

Embedded applications、Windows 3.1/早期 Macintosh (yield switch)、Windows 95 ME (yield/timer switch) 中简单的使用 Type II VM + Multiplexing

进行 Multiprogramming (Concurrency)——线程和 vCPU 是直接共享内存 (包括指令) 和 I/O 设备的，但越易共享越难安全。

Dual Mode Operation 两种运行层级

硬件至少用 1 bit 表示并提供两种运行模式：

Kernel Mode (Privileged/Supervisor Mode)

User Mode：Standard Libs & Applications 无法直接接触硬件、无法读写 OS Kernel memory、无法改变 Page table pointer、无法拒绝切换线程或 interrupt 等等

应用通过 Syscall (System call) 借助 OS 间接调用硬件，同样跨模式的操作还有 interrupts 和 exceptions (后文展开)

Memory Protection: B&B and Addressing

进程将 Base 设为其分配的最小映射地址，Bound 设为最大，保证任何内存访问满足：Base ⩽ address < Bound。从而实现 Isolation 并保护 Kernel

Load-time relocation：程序在加载时，把地址直接调整到它最后真正放置 (物理硬件上) 的位置。需要重定位 Loader，但执行时不一定每次访存都做动态加法

Address space translation：虚拟地址不等同于硬件地址，由 translation hardware 决定最后访问哪一个 physical address。运行时 (on-the -fly) 动态通过加偏移量计算物理地址

B&B Registers 必须是 Kernel Mode 的，由 OS 管理

假设 OS 要运行进程 P1，首先处于 Kernel mode (sysmode = 1)，OS 设置 B&B、user PC、registers；然后 OS 运行 RTU (return to usermode)，切换为 sysmode = 0，同时 CPU 开始执行 user program……但是如果要打断/调用硬件/切换上下文就需要切回 model，为了控制风险，有三种 transfer——

Syscall：由进程主动发起 OS 服务的特殊指令，例如 read()、write()、exit()、fork() (internal, active, synchronous)

Interrupt：由外部 Timer 或 I/O 设备等硬件独立于进程发起 (external, passive, asynchronous)

Trap/Exception：CPU 在执行进程的某条指令时检测到异常条件并触发控制转移到 OS Kernel，例如 divide-by-zero、page fault、invalid instruction、protection violation 等 (internal, synchronous) 。

安全起见，三者皆为 UNPROGRAMMED control transfer，即运行中的进程切换到 OS 无需也无法给出 Kernel “目标地址” 参数，进程无法随意指定跳到 kernel 的具体位置。OS 通过维护一张 Interrupt Vector 表，如果产生 i 打断码，CPU 查询 vector[i] 获得 interrupt handler address，然后进入 kernel 执行：intrpllandler_i () {...}。

Lecture3 Threads and Processes

Multiplexing: Concurrency 时间复用：并发

并发：假设 OS 要运行进程 P1，首先处于 Kernel mode (sysmode = 1)，OS 设置 B&B、user PC、registers；然后 OS 运行 RTU (return to usermode)，切换为 sysmode = 0，同时 CPU 开始执行 user program；Timer Interrupt 切回 model；随后 OS 先将 P1 的 CPU states (registers) 存…

切换上下文时，如何 “打包” 进程状态然后另存起来*?

PCB 进程控制块

OS Kernel 为每一个进程建一个数据结构 Process Control Block，Kernel Scheduler 建立一个保存所有 PCB 的数据结构并掌握把 CPU 切给各个进程的策略：

status 包括 Running/Ready/Blocked…；非 ready 状态下保存寄存器状态

切换上下文时，如何决定切给哪个进程-线程*?

Scheduler 调度器

*Mechanism (机制)：如何切换

*Policy (策略)：确保 Fairness、Latency 优化、Realtime Guarantees…

Threads

OS 切换进程实现并发，那为何进程还需要多个线程? 线程拥有什么资源/数据。

线程是独立的执行上下文，也是调度器可以独立调度的执行流，还是并发的机制 (Mechanism)，以及多核处理器并行的最小单位。

并行与并发的区别是前者 CPU 硬件上同时有多个线程运行，后者同时只有一个线程在 CPU 运行。所以单核无法并行，并发无需多核。

Multiprocessing = Parallelism = Simultaneous Execution = PLP

Multiprogramming = Concurrency = Overlapping Execution

线程存在的意义在于其 “独立” 性：

如果没有线程，calculate 要一直等着，因为前一个指令 (I/O) 是千万纳秒级别的；

但既然两个任务是独立的，分为两个线程 T1 和 T2，创造每个线程各自独占一个 CPU 的幻觉。这样将 I/O 作为独立线程，就可以隐藏其 Latency——线程相当于进程执行时的防堵车系统。

Multithreaded Process 多线程进程

当 C 程序被编译并运行后，该程序的进程被建立，进程中只有一个 initial thread 在其地址空间中，进程在运行期通过 Syscall 调用 OS 创建线程。这些线程都共享进程的地址空间、代码、堆、全局/静态数据，拥有自己的栈。

Syscall、Interrupt、Exception/Trap 都是由 User mode 转为 Kernel mode 的 UNPROGRAMMED 命令。你不会在 C 程序中手写它们。程序运行时通过 OS Library 提供的 API 发起 Syscall：

*C 程序 → libc / runtime wrapper → syscall instruction → kernel

C 程序默认从一个 main thread 开始。你不手动创建线程/进程，也不使用替你并行化的 runtime/library，就不可能有程序级 TLP；但 CPU 仍然可能在单线程内部利用 ILP、流水线、乱序执行、SIMD 等硬件并行。C 语言不会因为机器有 10 个核，就自动把你的顺序程序拆成 10 个线程。

三个重要的线程相关 API——pthread:

pthread_create(..., worker, arg)：相当于创建一个线程从 worker(arg) 开始执行

attr 包括栈大小，scheduling policy 等信息

create 后，join 不会再修改 pthread_t thread

线程函数里 return v；效果相当于隐式调用 pthread_exit(v) 来结束当前线程。

进程 (比如 main()) 的返回值只传整数状态码，就是因为不同于线程，进程间不共享内存地址，它们通过 socket/pipe 共享。

join 的第二个传参接收前序 exit 线程的传出数据，由于线程共享地址空间，使用指针的指针可以修改这个传出变量。

Fork-Join Pattern

Fork = Main Thread 把独立工作分出去并发 Work;

Join = 等这些工作全部完成，越过汇合点后 main 收集结果并继续。

问题：如果多个线程会读写进程的同一个变量，不加限制 (locking) 的话结果就不确定——Race Condition

Thread State

同一个 process 的 threads 共享：Code + Global Variables + Heap + I/O state/files + Open File Table + Address Space(B&B Register) + Page Table

但是每个 thread 私有：TCB(Registers (including SP/FP)、PC、Thread metadata) + Execution Stack

(Thread 私有状态除了 Stack 都装在 TCB (线程控制块) 中)

调用函数时，要暂存 “回来以后从哪继续执行”，这种地址即 Return PCs (RA) 通常存在 stack 中;

DFS ≡ Stack ≡ LIFO 出入栈天然适合处理函数递归调用的

BFS ≡ Queue ≡ FIFO

所有尚未返回的函数调用都需要保留各自的 RAs，通常保存在该线程自己的栈中

发生 thread context switch 时，OS 保存当前线程的 PC、SP、registers 等 CPU context；stack 本身仍然留在内存中，不需要把整个 stack 复制/保存一遍

(现代编译器不一定用 FP，通过 SP 固定偏移确定帧地址)

调用关系：exit—入栈→A(1)→B→C→A(2)—出栈→C→B→A(1)→exit

线程运行时 SP 在 CPU 寄存器中，指向内存中该进程的栈顶；

线程切换时 SP 等 execution states 被存到 Kernel Memory-TCB 中

*Scheduler 真正切换的执行上下文通常是 thread

Process 更多代表：资源/保护/地址空间的容器

Thread 更偏向于：CPU 执行/调度的基本单位

Lecture4 System Programming: Processes

Interleaving & Nondeterminism

硬件上多条执行流 (Threads) 交错执行注定了程序必须适应任何 schedule——

scheduler 可以按任意顺序运行线程；

scheduler 可以在某线程运行到任意时刻切换；

因而测试 concurrent programs 很困难。

Independent Threads：互相不共享任何状态 (代码层面：不修改同个变量)，反之就是 Cooperating Threads，如无有意设计可能得到多种 interleaving 执行结果 (Race Condition)。

如下情况 x 的执行结果有多种可能：1,3,5

类似的，非原子操作 (non-atomic op.) 可能执行出多个结果：

Increment 对于 CPU 相当于 read、add、write 三个指令，在指令间切换的导致结果可能是：2 或 3

共享数据结构则更危险，尤其是操作步骤多的树，可能在中间步骤切换导致出现结构 (指针/节点) 损坏或得到不合理结果。

Scheduler 是 nondeterminism 的，故要在设计程序时保证稳定准确 (deterministic)。

Relevant Definitions

Synchronization：协调多线程执行 (主要是共用数据)

Mutual Exclusion (Mutex)：一种 synch.，某件事情同时段只能有一个线程做，用于保护关键代码

Critical Section：同时段最多允许一个线程执行的代码区域 (护持保护的代码)

Lock：同时段只能由一个线程持有的 object，用于实现 Mutex。提供两个原子操作——

lock.acquire()：等 lock free 后，调用该指令的线程占有 lock (hold the lock)；

lock.release()：占锁的线程调用以释放 lock。

在 CS 前后加 lock 指令使后续线程无法打断正在执行的线程。因此不会 overlap。

(暂不讨论如何实现 lock，但显然从线程角度，它绝不可被拆成多个步骤：.acquire must be atomic)

OSlib Pthread 里实现真正的 Mutex:

Processes …… Again

Everything outside of the kernel is running in a process

比如在终端里输 ls，kernel 不会凭空生成一个 list directory 进程——实则 shell 本身就是一个进程，向 kernel 发起 Syscall 并创建 ls 进程!

也就是说：进程可以创建进程，进程可以管理进程

Process Management APIs (Unix)

exit 结束当前 process (类比 void pthread_exit)

fork 复制当前 process，产生一个新的 child process (类比 int pthread_create)

注意 Thread 的 fork-join pattern 泛指 “分叉产生并发任务”，和 Unix Process API 的 fork() 不可混为一谈

exec 把当前 process 正在运行的 program 换掉

wait 等待 process 结束 (类比 pthread_join)

kill 给另一个 process 发 signal (interrupt-like-notification)

sigaction 设置收到某种 signal 后怎么办

exit()

exit() 终止整个进程，接收一个整数进程退出状态码：void exit(int status);

由于 main() 函数是程序入口，它返回后程序结束，OS Library 会帮用户 exit() ——exe 文件的入口 (entrypoint) 在 OS lib 中，故它会调用 main 并在其返回后调用 exit。

这解释了为何 main() 函数永远输出 int：int main()

这和线程中的线程函数 (thread start routine) return 相当于隐式 pthread_exit 结构上形成对应关系

return 代表退出当前函数，而 exit() 可以在程序调用栈的任何位置直接终止整个进程。比如 main 调用函数时出现异常/故障/缺失：

fork()

pid_t fork()：复制当前进程，对应所谓 “进程可以创建进程”

新进程拥有不同的 PID，并初始拥有一个线程

fork() 后，产生的子进程得到 parent 在 fork 那一刻的进程状态副本，包括 Address Space、File Descriptors、代码数据堆栈寄存器 PC 等。 但 OS 对两者虚拟地址 mapping 不同，实际物理地址分离，后续读写互不影响

```
printf("A"); : fork(); : printf("B");
```

对于上述代码，输出结果一般是 A B B，因为 fork 后父子进程都从同一位置继续执行 (谁先输出不确定)。

fork() 最精妙的设计在于用 2 个返回值 (pid_t PID) 识别 parent&child!

比如 cpid = fork();

如 cpid > 0，可以确定这段代码一定在 parent 进程中执行，且 cpid 就是子进程的 PID

如 == 0，则可确定在新产生的子进程中运行

如 < 0，那没有成功 fork 出子进程，只存在原进程

parent 进程调用 fork()，返回时父子进程都会返回，得到两份 cpid，所以 fork() return TWICE

parent 需要知道 child PID 以方便对它 kill、wait，而 child 只需通过 getpid() 就可以查自己的 PID——

*parent 得到 Child 的 PID；Child 得到 0 这个 “我是 Child” 这个标记。

输出结果有两种 (由于调度器并发)：

```
Parent pid: 100 Parent pid: 100 : [100] parent of [101] [101] child : [101] child [100] parent of [101]
```

这种 race 和线程的不一样 (不存在 shared memory data race)，因为进程中的执行顺序是固定的，但父子两个进程的线程可以并发交替：threads concurrently execute——两个 process 的输出顺序可能交替。

创造线程 vs 创造进程

在各个进程的输出间加 sleep() 可以让输出更容易交错，但不能保证如此，因为 sleep 只是让进程/线程暂停，并无 synchronization 的功能。

(Windows 使用 CreateProcess() 代替 fork()，接口更复杂)

exec() & wait()

execv("/bin/ls", args)：标识把当前进程执行的程序替换成 ls，不创造新进程——PID 不变但代码数据堆栈都变。

```
char *args[] = {"ls", "-l", NULL}; : execv("/bin/ls", args); : /* execv doesn' t return when it works. ; So, if we got here, it failed! */ : perror("execv"): : exit(1);
```

exec 正常运行时进程运行的程序从当前程序切走，所以如果它返回且执行下一行，必然出错了。

显然，shell 进程不能直接 execv() 任何指令，否则 shell 自身的程序就被干掉了。必须 fork() + child execv()。并且 shell 为了显示指令 (如 ls) 输出结果，必须等待这个子进程运行结束。整个逻辑串起来：

假设 parent 和 child ID 分别为 100 和 101，前者先输出 [100] parent of [101] 然后等待；后者输出 [101] child 然后输出当前目录的文件列表；正常输出结束后，前者输出 [100] bye 101(0)。 实际输出时前两个输出顺序不保证。

wait(&status)：等任意一个 Child (最早结束者)

waitpid(pid, &status, 0)：可以指定等哪个 Child

kill() & signaction()

Signal 是发给进程的非硬件异步 (asynchronous) 通知

向 OS 注册一个 signal handler；“如果我收到 SIGINT (CTRL-C)，不要按默认方式处理，请调用 signal_callback_handler()”

如果不注册 handler，按照默认，进程直接死。

SIGINT (interrupt)：CTRL-C

SIGTERM (terminate)：shell 默认 kill PID

SIGSTOP (stop/pause)：CTRL-Z

SIGKILL (进程不能捕获、不能忽略、不能自定义 handler) 可靠的强制终止

Threads：共享方便，但 synchronization 困难

Processes：隔离更强，但 communication 更麻烦

Lecture5 Files / Pipes / Sockets

Files & FD / OFD

POSIX 是 API 标准；Unix 将 I/O 统一抽象为 byte stream；inode 记录文件元数据，目录建立路径到 inode 的映射。

FD 是进程局部整数；FD table → Open File Description (OFD: offset/status flags) → inode；dup()/fork() 可共享 OFD 与 offset；独立 open() 同一文件产生不同 OFD。

open/read/write/close 使用 FD；fopen/fread/fwrite/fclose 使用 FILE* buffered stream；fread/fwrite 返回 item 数，read/write 返回 byte 数；短读写需循环处理。

lseek 改 OFD offset；pipe/socket 不可 seek；O_APPEND 每次写入在 EOF 追加；fflush 刷用户缓冲，fsync 请求持久化。

Process IPC / Pipe

pipe(fd)：fd[0] read，fd[1] write；fork 后关闭不用的端；所有 write end 关闭后 read 返回 0 (EOF)；无 reader 时 write 触发 SIGPIPE/EPIPE。

dup2(old,new) 令 new 指向同一 OFD，常用于重定向；stdin=0 stdout=1 stderr=2；shell >、2>、|。

Sockets / Network

server: socket→bind→listen→accept (新连接 FD，listen FD 保留)；client: socket→connect；双方 send/recv。

TCP 是可靠有序 byte stream，不保证消息边界；recv 可能短读，返回 0 通常表示对端有序关闭；UDP 是 datagram。

IP 标识主机，port 标识服务端点；127.0.0.1 localhost；网络字节序 big endian，htons/htonl；thread pool 可限制线程创建开销。

CSC3150 Lecture6 Concurrency

Threaded programs must work for all interleavings of thread instruction sequences.

Revision of Threads

多线程并发的速度和资源复用的优势隐含着 race condition 的风险。

易混淆概念：

只有 global/static 变量会发生 race，因为它们处于同个进程中共享的堆上！比如 for 循环条件中的自增项 i 就在两个线程各自的栈上，不会冲突。

*多个 execution sequences 的执行可能交错（interleave）—— 单核也可以错。所以多线程需要 synchronization（同步机制）协调，例如 mutex、semaphore、atomic operation 等。

如此，程序结果依赖多个并发执行之间不可控的 timing / interleaving —— Race Condition

Tradeoff of Threads

为什么明知有风险还要线程*?

由于线程的 race condition，两个人同时向同个账号汇款可能导致信息覆盖。不是说线程本身危险，而是 并发和共享可变数据 本身就潜在风险。

Atomic Operations

Mutex and Locks

```
pthread_mutex_t lock; ; pthread_mutex_init(&lock, NULL);
pthread_mutex_t lock; ; pthread_mutex_init(&lock, NULL); ; deposit(acct Id, amount) { ; acquire(&lock); ; acct = get_account(acct_id); // ; acct->balance += amount; // Critical Section ; store_account(acct); // ; rel…
```

Designing of Locks

```
leave note_A; leave note_B; ; while (note_B) { if (no_note_A) { ; do nothing if (no_milk) { ; } buy_milk(); ; if (no_milk) { } ; buy_milk(); } ; } remove note_B; ; remove note_A;
```

*问题是这个不对称解法过于复杂，在现实机器中经常存在成百上千条多线程，怎么谁记每个线程不同的上锁协议？对代码长度负担太大，且不对称所需的等待会消耗大量 CPU 时间（ busy-waiting）。

Leslie Lamport's Bakery Algorithm (1974) 证明了理论上可以用 shared memory 的读写设计复杂的软件 mutual exclusion protocol…但这太复杂了。

CSC3150 Lecture7 Synchronization & Lock

Solution1: Disable Interrupt Lock

这只适用于单处理器，因为每个核独立控制 interrupt 开关

Naive Disable Interrupt

Disable on Lock-ops

Missed Wakeup：唤醒发生时，等待者尚未正确进入等待状态，因此错过了本应唤醒它的事件。

Solution2: Atomic Read-Modify-Write Lock

多核 CPU 上禁用本核中断不能阻止其他 Core

需要更通用的硬件支持：READ-MODIFY-WRITE 不可分，以下为几种原子指令的 C 代码逻辑描述：

Test&Set

*这段 C 代码只是语义描述，真正的 T&S 是硬件支持的原子操作，无法用 C 1oad/store 实现

它读取内存中的旧值，把内存设置为 1，然后返回旧值（0：原来没锁；1：原本已上锁）

Compare&Swap (CAS)

*IFF 内存保持我预期的状态，才允许修改，这在并发编程中非常有用——CAS 能解决 Race Condition，当不符合预期说明丢失更新（Missed Update），并触发重新 read

*CAS 保证不会基于已经过期的预期值错误地覆盖别人的修改，不使用传统 Mutex 协调共享多线程的更新

Swap

Spinlock

无需 Kernel 读写，存在于 userspace 的简单原子自旋锁实现：

```
int lock = 0; // Interface: acquire(&mylock); release(&mylock); ; void acquire(int *lock) { ; while (test_and_set(lock)) { ; } ; } ; void release(int *lock) *lock = 0;
```

检查和设置锁状态发生在同一个 Atomic Operation 中，所以两个线程不可能同时通过这一检查

Spinlock 中，等待线程 *lock = 1 一直循环 while (T&S)，不会主动进入 Blocked 状态，这种等待方式叫 Busy Waiting

每次 T&S 循环都有尝试 write 操作，在多核情况下不仅浪费等待线程的 CPU，还可能增加缓存一致性（Cache Coherence）流量，浪费 network bandwidth。

要求设计同步算法时，除非明确允许自旋，否则应优先考虑能让等待线程睡眠的方案，避免 Busy Waiting。

Better T&S Lock

结合前两种方案优点：只在极短的内部操作中使用 Spinlock，真正等待锁的线程进入 Sleep

变量 guard 负责保护 Lock Implementation 本身的 Mutex

acquire 的关键是 Sleep 和 Release Guard 必须不可分割地协调完成，不可写成两个独立操作，否则又可能出现 Missed Wakeup

release 中若等待队列有线程，并不直接将 mylock 设置为 FREE，因为锁可以直接交给等待线程（所有权交接时锁保持 BUSY），只有队空时释放锁。这种设计下等待线程不需要一直消耗 CPU 自旋

Linux Futex

Futex = Fast Userspace Mutex

纯 Spinlock 不用进入 Kernel 但等待时持续消耗 CPU；Kernel Blocking Lock 可以 Sleep 但 频繁 Syscall 有额外开销—— Futex 希望 Lock FREE 时在 Userspace 直接获取，Lock BUSY 时进入 Kernel 以 Sleep。

uaddr 指向一个 userspace 中的 32-bit 值

FUTEX_WAIT：如果 *uaddr 仍等于预期值 val 则进入 Sleep 直到 FUTEX_WAKE（为避免 Missed Wakeup）

FUTEX_WAKE：唤醒最多 val 个等待线程

timeout：指向一个 timespec structure 以设置 futex_op 的超时时长的指针

futex 不向 libc 暴露，它被运用于 pthread implementation 中

Futex T&S Lock

```
int mylock = FREE; ; acquire(int *thelock) { ; while (test_and_set(thelock)) { ; futex(thelock, FUTEX_WAIT, 1); ; } ; } ; release(int *thelock) { ; *thelock = 0; ; futex(thelock, FUTEX_WAKE, 1); ; }
```

Lock FREE 时，线程没有调用 Futex 就在 userspace 获得锁（Uncontended Lock → Userspace Fast Path）

Lock BUSY 时，通过 Futex 控制 Kernel sleep 以排队（Contended Lock → Kernel Slow Path）

发生竞争时才需要进入 Kernel，所以可以同时获得：无竞争时的低开销、发生竞争时的阻塞等待、无长期 Busy Waiting、支持多核 CPU。但 release 仍需要进入内核，即使实际上没有等待者

实际的 Futex-based Mutex 可以进一步优化，避免大量不必要的系统调用。

Semaphores 信号量

如果一个线程必须等待另一个线程完成某件事，只有 Lock 通常不够

Programs：使用共享资源。例如多线程应用

Higher-level API：同步与协调。例如 Locks、Semaphores、Monitors

Hardware：提供原子操作能力。例如 Load/Store、Test&Set、CAS、Disable Interrupts

Producer-Consumer Problem

Producer（生产者）：向 Buffer 中添加数据。

Consumer（消费者）：从 Buffer 中取出数据。

不希望 Producer 和 Consumer 必须严格一对一地交替执行，Buffer 让两者可以不同速度工作

Buffer（缓冲区）：生产者和消费者共享的存储空间。只需要保证：

Buffer 不会溢出，不会从空 Buffer 中取数据。

多个线程不会同时破坏 Buffer 的内部状态。

*Producer 必须在 Buffer Full 时等待。同样，如果 Buffer 为空 consumer 也不能 dequeue。这就是 Bounded Buffer Problem（有界缓冲区问题）

Circular Buffer 循环缓冲区

```
typedef struct buf { ; int write_index; ; int read_index; ; *entries[BUFSIZE]; ; } buf_t;
```

write_index：下一个写入位置

read_index：下一个读取位置

entries[]：存储数据的数组

BUFSIZE：Buffer 的容量

```
entries[write_index] = item; ; write_index = (write_index + 1) % BUFSIZE; ; item = entries[read_index]; ; read_index = (read_index + 1) % BUFSIZE;
```

这里 % BUFSIZE 是为了循环使用数组，比如 BUFSIZE = 4，write_index = 3 时入队，写指针会从最后一个位置 3 回到第一个位置 0

判断 full/empty：read_index == write_index 无法确定是满还是空，一般要维护 count 确定满/空。

Circular Buffer with Lock

Producer 操作 Buffer 前先获取锁。

Consumer 操作 Buffer 前也先获取锁。

保证同一时间只有一个线程操作 Buffer。

但实际上存在严重问题：假设 Buffer 已满，Producer 获得锁并 busy wait 缓冲区出现空位，但需要 Consumer 获得锁才能腾出空位，于是形成循环等待 Deadlock，程序可能永远无法继续。

```
while (buffer full) { release(&buf_lock); acquire(&buf_lock); } ; while (buffer empty) { release(&buf_lock); acquire(&buf_lock); };
```

那么这样 Consumer 就有机会获取 Lock 并消费数据，因此之前的循环等待被打破了。 但是又有新问题：while 仍然在不断尝试，即使 Buffer 长时间没有读写

单核下 Producer 可能重新获得锁，Consumer 需要等待调度机会，造成不必要的 CPU 消耗。

多核下 Producer 可能在一个 Core 上不断检查，Consumer 在另一个 Core 上工作，产生不必要的锁竞争和缓存一致性开销。

*虽然没有 Deadlock 但还存在 Busy Waiting 问题。希望 Buffer FULL 时立即 Producer BLOCK，并切给 Consumer。

Semaphore

Semaphore 信号量 是 UNIX 的一种通用同步原语（Sync. Primitive），属于一种泛锁。Semaphore 内部有一个非负整数，但它不是普通 int，不能随意修改或读写，只可以：

通过 Down()/P() 等待 Semaphore 大于 0，然后原子自减 1（不允许减到负数）；

通过 Up()/V() 原子自增 Semaphore 的值，并在需要时唤醒等待线程（比如 P() 在等 >0）。

P 和 V 必须是 Atomic Operations，并避免 P() sleep 等待不能 miss V() wakeup

用法一、Semaphore 实现 Mutex：初始值为 1，这种用法称为 Binary Semaphore（二进制信号量）

```
P(&mutex); // 相当于获得锁 ; V(&mutex); // 相当于释放锁
```

用法二、Scheduling Constraints

假设要保证 A 与 B 线程先后有序执行，信号量初始值为 0，后执行者运行 P()，先执行者运行完成后执行 V()。

这就是 Semaphore 的重要特性：处于 0 时，V() 需先于 P() 发生

```
Semaphore done = 0; ; ThreadJoin() { ; P(&done); ; } ; ThreadFinish() { ; V(&done); ; }
```

即，可以用信号量表达“现在有多少个可用资源”

Bounded Buffer

回到 Producer-Consumer Problem，需要三个 Correctness Constraints 信号量：

Consumer 必须在 Buffer Empty 时等待

Producer 必须在 Buffer Full 时等待

同一时间只能有一个线程操作 Buffer Queue

*前两个是 Scheduling Constraint，后一个是 Mutex

```
Semaphore full_slots; //有多少可消费位：初始0 ; Semaphore empty_slots; //有多少可写入位：初始BUFSIZE ; Semaphore mutex; //Buffer Lock：初始1
```

Producer-Consumer Solution:

P() 顺序不可变：若交换申请位和获得锁，可能出现 Deadlock 循环等待。必须先等待资源可用，再获取保护共享数据结构的 Mutex。

与 P() 顺序不同，交换这两个 V() 的顺序并不必然造成 Deadlock，在正确 Semaphore 语义下也可保持同步正确性，因为 Consumer 即使被唤醒，也仍需要等待 Mutex。但这可能造成不必要的竞争。

Semaphore 相比单纯使用 Load/Store 或 Lock，是很大的进步。因为它可以同时处理：Mutual Exclusion 和 Scheduling Constraints，但问题也正来自这里：Semaphore 有两种不同用途，却使用完全相同的 P/V 接口

仅仅交换两个 P() 就可能造成 Deadlock，这引出了下一讲的主题：Monitors（管程）。

CSC3150 Lecture8 Monitors

在用 3 个 Semaphore 实现 Buffered Producer-Consumer Lock 中，我们发现仅仅交换两个 P() 就可能造成 Deadlock —— 我们希望有一种更清晰的抽象把 Mutex 和 Scheduling Constraints 分开表达，这就是 Monitor 的设计思想。

Monitors 管程

*Monitor 管程抽象 = 一个 Lock + 零个或多个 Condition Variables，用于协调对共享数据的并发访问。

其中锁负责维护 Mutex；条件变量负责维持 Scheduling 限制

Monitor 并不一定是某种独立的硬件结构或特殊对象，它首先是一种并发编程的组织方式（Programming Paradigm）。Java 等语言原生提供 Monitor 机制；其他语言可以通过 Lock + Condition Variable 实现。

条件变量

Condition Variable 是一个等待队列，用于存放等待某个条件成立的线程 —— queue for threads in critical section to sleep

例如 Condition not_empty：代表可能存在一些 Consumer 线程正在等待 Buffer 变成非空，它们并非 Busy Waiting，而是 BLOCKED Sleeping，当 Producer 添加数据后，可以通知它们。但是 Condition Variable 本身不存储条件是否成立。

对比 Semaphores：

Semaphore 内部有计数值；而 Condition Variable 没有类似的资源。

Semaphore 通常不使用普通 Mutex 而利用自身操作（二进制信号量）实现；Monitor 通常配合使用 Mutex + 条件变量。

Condition Variable 允许线程在临界区内（持有 Mutex 情况下）调用 wait()，因为它能在进入等待状态时 原子性地释放锁，从而避免持锁睡眠造成的 Deadlock，也避免释放锁与进入等待之间发生的 Missed Wakeup。这也是为什么不能用普通的 release() + sleep() 替代 Condition Variable；

Semaphore 不能在 critical section 中等待（Deadlock）。

条件变量的操作（必须持锁者才能执行）：

Wait(&lock)：原子地释放 Lock 并进入等待；返回前重新获取 Lock

Thread A：acquire(mutex) → 检查条件 → xxx 条件不满足 cond_wait(&xxx) → 原子释放 mutex 并进入相应条件的等待队列 → BLOCK → 被 cond_signal 唤醒 → 等待重新获取 mutex → 获得锁后 cond_wait() 返回，继续执行

因此有一个非常重要的性质：cond_wait() 返回时，调用线程已经重新持有 Lock。不需要再获取锁，否则对普通的非递归 Mutex，可能造成自己等待自己释放锁的 Deadlock。

Signal()：唤醒一个等待线程（如果存在）

cond_signal(&xxx) 表示如果有线程正在等待 xxx，唤醒其中一个，不保证出队顺序！唤醒谁由实现和调度策略决定。若没有等待线程通常不会留下任何通知或值。

Broadcast()：唤醒所有等待线程

cond_broadcast(&xxx) 唤醒所有等待 xxx 的线程，但它们仍需重新竞争获得一个 Mutex。

Infinite Synchronized Buffer

通过 Monitor 实现无限容量缓冲，那么 Producer 永远不需要等待空位，Consumer 仍然需要等待数据。因此我们只需要一个 Condition Variable，它对应 Semaphore mutex。

为什么不用 if (is_empty(&queue))?

它们最关键的区别是执行 signal() 后，被唤醒线程能否立即获得 Monitor 的执行权——检查一次还是每次醒来都再检查一次？而这取决于调度的类型。

Hoare Monitor：当一个线程执行 signal()，等待线程立即获得 Monitor，实现需要复杂的执行权交接机制，但可以使用 if/while 逻辑

if 相当于条件成立就等待一次，醒来后不再检查

Mesa Monitor：执行 signal() 的线程不会立即让出 Mutex，被唤醒的线程只是进入 Ready Queue，等待未来被调度（期间其他线程也可竞争锁），只能用 while 逻辑

while 不是 Busy Waiting！相当于每次条件不满足，线程都会通过 cond_wait() 释放锁并阻塞。每次醒来都重新检查，若条件仍不满足，就继续睡眠

Mesa Monitor 的 Signal 只表示某个条件可能已经发生变化，不保证被唤醒线程真正运行时条件仍然成立。

*大多数操作系统采用 Mesa-style scheduling

Bounded Buffer with Monitor

这次不使用三个 Semaphore，而是三个 Monitor 来构建有限缓冲同步区

这比 Semaphore 版本更加直观，条件判断部分直接表达：

P：当 Buffer 满时，等待 Buffer 不再满；C：当空时，等到不再空。

cond_wait() 让线程原子释放 Lock 并进入等待，所以其他线程可以进入临界区，这就避免了使用普通 Mutex 时的 Deadlock。

Mesa Monitor Conclusions

```
acquire(&lock); ; while (need_to_wait) cond_wait(&CV, &lock); ; release(&lock);
```

改变状态并通知其他线程：

```
acquire(&lock); ; cond_signal(&CV); ; release(&lock);
```

检查共享条件时，必须持有对应的 Lock，否则检查条件与进入等待之间可能出现 Race Condition

cond_wait() 会自动释放 Lock，并在返回前重新获取 Lock，可在临界区中等待其他线程改变共享状态

使用 while 重新检查条件，而不是假设被 Signal 后条件一定成立，Mesa 不保证被唤醒线程立即执行

Semaphore 的 V() 和 Condition Variable 的 Signal 并不等价：因为信号量通过计数保存许可，而管程不保存通知。如果 signal() 执行时无等待者，通知就会丢弃；后来的 cond_wait() 不会因为之前执行过 signal() 而返回。因此条件变量必须与一个受 Lock 保护的共享状态条件 while 配合使用。

Readers-Writers Problem

相比上一部分的 Producer-Consumer，读者-写者问题的并发同步逻辑更复杂

对于一个共享数据库，存在两类线程：

Reader 读者：只读取数据，不修改

Writer 写者：可以读取并修改数据

只用 Mutex 可以解决这个问题……但这种方案有一个明显的缺点——无限制互斥：多个 Reader 都只读，但它们也无法同时访问，时间复用率（multiplexing）低。

三条 Correctness Constraints：

① Reader 可以在没有 Active Writer 时访问数据库（读者间不互斥）

② Writer 只能在没有 Active Reader，也没有 Active Writer 时访问数据库（写者全局互斥）

③ 同一时间只有一个线程可以修改同步状态变量

四个 state variables 状态变量（受锁 lock 保护）：

```
int AR = 0; // Active Reader ; int WR = 0; // Waiting Reader ; int AW = 0; // Active Writer ; int WW = 0; // Waiting Writer ; Lock lock; // 保护状态变量 ; Condition ok_to_read = NIL; // 等待读的队列 ; Condition ok_to_write = NI…
```

通常采用 Writer Priority（写者优先）策略，即当已经有 Writer 正在等待，新 Reader 也必须等待。否则可能发生 Starvation：Reader 源源不断，Writer 被指死无法执行的情况

因此，Reader 的等待条件不是 AW > 0 而是 AW + WW > 0 ——

*不需要在整个数据库读取期间一直持有 Mutex，这样才能允许多个 Reader 同时读取数据库。但前提是 Reader 只读，Writer 也必须遵守同一套访问协议，否则仍然可能出现数据竞争。

Writer：AW 保证 Writer 之间的互斥，而 WW 用于实现 Writer Priority ——

Readers 可能发生 Starve 吗？因为使用 Writer Priority，假设写者不断到达，就永远无法读——写优先并不等于 Fairness，只是避免 Writer Starve。

如果删除 Reader 退出时的 AR == 0 判断？通常不会直接破坏互斥正确性，因为 Writer 有 while(AR+WR > 0) 保护。但造成不必要的唤醒、锁竞争和上下文切换。

如果把给 write 的 signal() 改成 broadcast()？正确性通常仍然可以维持，但会产生不必要的唤醒和锁竞争，这类现象称为 Thundering Herd（惊群）。

如果只用一个条件变量 ok 而非分别处理读写？问题是 signal 不保证唤醒哪一种线程，可能导致系统没有线程继续推进，就必须用 broadcast 让所有等待线程重新检查自己的条件。代价是更多不必要的唤醒，不如两个条件变量高效清晰。

为何没有 R/W Race Condition：读者的 AR++ 和写者的 AR==0 是在同把锁保护下执行的，不可能出现 Reader 认为已经获得读资格，同时 Writer 认为没有任何 Reader 正在读的可能。

Concurrency Programming Tips

C 语言

异常或提前返回时容易忘记释放 Lock：

发生错误时不能直接退出函数，必须确保每条退出路径都释放 Lock。

必须 catch 关键区域（Mutex 区域）的所有 exception

C++

当函数结束、离开作用域或发生异常导致栈展开时，lock_guard 的析构函数 destructor 会自动释放 Mutex

所以不需要手动写 global_mutex.unlock();。

Python

with 关键词自动获取 lock，离开代码块时自动释放 release()；即使代码块中发生异常，也会执行相应的退出清理操作。

Java

synchronized 方法会获取对应对象的 Monitor Lock，同一个对象上的其他 synchronized 实例方法，需要竞争同一把锁。

Java 还提供 wait()、notify()、notifyAll()；对应 Monitor wait、signal、broadcast。

Java 的 wait() 也会释放对应对象的 Monitor Lock，并在返回前重新获取。

CSC3150 Lecture9 现代计算机系统的并发

Concurrency Issues Today

之前学习并发时主要考虑多线程共享数据的 Race Condition，真实软件中的并发 Bug 更复杂。

*Order Violation（执行顺序违反）：OS 调度器不保证两个线程按程序员希望的顺序执行。不能简单的用 sleep()，常见解法是 Monitors-Condition Variable。

*RaceBench（CCS 2023）在真实程序自动注入可触发并发 Bug，用来测试并发分析工具。真实并发 Bug 往往很难稳定复现，RaceBench 会先记录 Execution Trace 即多线程交错情况再进行 Bug Injection。

注入代码分为：Prologue 设置 Bug 所需的状态、Precondition 判断是否满足触发条件、Bug Core 执行可能产生并发错误的操作。可稳定复现。

Machine Learning Parallelism

Data Parallelism:

Model Parallelism:

PipeDream — Pipeline Parallelism for DNN Training

*PipeDream（SOSP 2019）的核心思想：结合数据并行、模型并行和流水线调度，让多个 GPU 尽可能持续工作，提高模型训练速度，减小传输开销，增加硬件资源利用率。

PipeDream：对于 Model Parallelism，当 GPU 1 处理完 Batch 1，并把结果交给 GPU 2 后，GPU 1 不必空闲立即处理 Batch 2…… 速度约数据并行 5.3 倍。

Pipeline Partitioning & Load Balancing

Minimize Communication：减少阶段之间的数据传输时间。某些层间可能需要传输大量中间结果，所以 最优划分不一定是每个 GPU 分配相同数量的层，需结合考虑负载与通信

1F1B Scheduling

Weight Stashing

每次 Backward 完成后会更新模型 W，导致还未完成的 Backward 参数与其对应 Forward 不一致，出现 Weight Version Mismatch —— 模型参数版本不一致会破坏梯度计算的正确性

为什么增加 Micro-batch 数量不能消除 Stage 3 的瓶颈?

为什么 1F1B 可以先执行 batch 5 的 Forward，再执行 batch 2 的 Backward? 不违反计算依赖吗?

为什么 Weight Stashing 能保证梯度计算一致性，却不能保证所有 batch 都使用最新参数?

Other Parallelisms

Pipeline Parallelism 拆分 Layers，Tensor Parallelism 拆分 Layer 内部的计算，将同一层内部同一数据的矩阵运算拆分到多个 GPU 上执行（两个 GPU 就能并行计算同一个神经网络层）

Expert Parallelism 主要用于 Mixture of Experts（MoE，混合专家模型），普通神经网络让输入经过固定的网络结构；MoE 则包含多个 Expert 专家子网络（分布在多个 GPU 处理），一个 Router 根据输入决定使用哪些 Expert，将 Token 发送到不同 GPU，并收集 Expert 的输出。

LLM: DistServe

从神经网络的 Training 转向 Inference 推理。

LLM 推理的两个阶段:

Prefill 预填充阶段

“请详细解释什么是 Operating Systems，以及它们如何管理 Memory、Process 和 Threads。”

这段 Prompt 包含多个 Tokens，模型首先并行处理整个输入序列，计算各层的中间表示，并建立后续生成所需的 KV Cache。

一次处理多个输入 Tokens;

计算密集，包含大量矩阵运算;

通常具有较高的计算并行度。

指标 TTFT：Time To First Token 从请求到达服务系统到第一个输出 Token 产生所经历的时间。TTFT 越低，用户越快看到模型开始回答

Decoding 解码阶段

Prefill 完成后，模型开始逐个生成输出 Token。Token 1：操作、Token 2：系统、Token 3：是、Token 4：一种……

在标准自回归逐步生成式推理中，每一步生成都依赖之前已经生成的 Tokens。无法简单地把同一个请求的所有输出 Tokens 完全并行生成，但不同请求的 Decoding 可以通过批处理并行执行。所以解码阶段更易受到显存带宽限制。

指标 TPOT：Time Per Output Token 每个输出 Token 的平均生成时间，不包含首 Token 的等待时间。

Disaggregting Prefill and Decoding：传统 LLM Serving 系统可能让同组 GPU 同时承担 Prefill 和 Decoding 这两种具有不同计算特征的任务。这会导致：Prefill 干扰 Decoding，增加 TPOT；反之正在运行的 Decoding 也可能影响新请求的 Prefill 调度，使 TTFT 增加。

DistServe 将 Prefill 和 Decoding 分配给不同的 GPU 资源池，但 Prefill 完成后，必须将 KV Cache 传输给 Decoding GPU。若 KV Cache 大/网络带宽不足，传输开销可能抵消分离带来的收益，DistServe 综合考虑计算资源分配、KV Cache 传输和服务延迟要求。

CSC3150 Lecture10 Scheduling Basics

Basic CPU Scheduling

*Scheduling 调度：OS 决定哪些线程在什么时间获得资源使用权（并将其从 ready queue 退队）

Scheduler 主要决定 READY 与 RUNNING 的转换；不过线程由于 I/O、Lock、Condition Variable 等因素 BLOCKED 后调度器也需要选择其他可运行线程。

程序通常在 CPU Burst（线程连续使用 CPU 进行计算的一段时间）和 I/O Burst（线程执行 I/O 操作并等待完成的阶段）之间交替执行。

CPU Burst 的长度分布通常偏向较短的 Burst；且经常可能被打断（preemptive scheduling）因此线程连续占用 CPU 的时间不一定等于其自然 CPU Burst 长度。

不是每次 read() 必然 Block，例如数据在缓存中时操作可能很快完成。

I/O Burst 不意味着 CPU 不参与 I/O，只是线程不占 CPU，但系统调用本身仍需 CPU 执行 Kernel 代码

反之 CPU-Burst 也不是完全不执行 I/O。

*Scheduling Policy Goals:

Minimize Response Time：最短响应时间（从任务开始到系统首次响应的时长）

Maximize Throughput：最大吞吐率（单位时间内完成的任务数量）

响应时间最小化可能通过更频 Context Switching 却降低系统吞吐量——两者不一定正相关

吞吐率最大化通常包括 1、缩小 overhead (e.g. context switching)；2、提高硬件使用效率/复用率（CPU，disc，mem etc.）

Fairness 公平性：不同线程或用户应以某种合理/公平的方式分享 CPU，比如防止某线程极端 Starvation。

*更短响应时间通常导致系统更不公平

Some Scheduling Schemes ——

First-Come First-Served (FCFS) scheduling

FCFS 调度算法也叫 FIFO 或 Run Until Done，相当于先到（Ready Queue）先得 CPU，典型的 非抢占式调度 Non-preemptive（不存在 Timer 打断）

在早期系统中被调度的线程会运行到终止，无视 I/O 等待;

现在一般指占有 CPU 直到 Terminate、Block 或 Voluntarily Yield。

*Waiting Time = Completion Time − Burst Time：平均等待时间为（0+24+17)/3 = 17 平均完成时间点为（24+27+30)/3 = 27 ——发生了 FCFS 典型缺点：Convoy Effect（车队效应）：多个短 CPU Burst 的线程被迫等待前方长 CPU Burst 的线程，导致等待时间增加、系统资源利用率下降。

去你的“护航效应”，你这唐 B 翻译去跟“套接字”一起吃达伽去罢

优点：实现简单、无需复杂优先级计算、无 Timer 抢占带来的频繁切换（Throughput ↑）

缺点：Convoy Effect（Throughput ↓）、对短任务不友好（Fairness ↓）、平均等待时间高度依赖任务顺序

Round Robin (RR) scheduling

RR 调度法在 FCFS 的基础上给每个线程分配固定长度的 CPU 时间片 q（Time Quantum）限制其最多连续执行上限，若在 q (ms) 内未 Block/Terminate/Yield，则 触发 Timer Interrupt，Thread RUNNING → READY 然后入队到 Ready Queue 末尾。

对于宽 n 的 Ready Queue，没有线程等待时间 > (n-1)/q ms。

所以 RR 是一种基于定时器的抢占式调度 Timer-based Preemptive Scheduling

Kernel 实现 RR = FIFO Queue + Timer Interrrupt + Synchronization

Quantum 太大 → RR 逐渐接近 FCFS（Response Time ↑，硬件使用率 ↓）；Quantum 太小 → Context Switch overhead 增加（Throughput ↓硬件使用率 ↑）

平衡响应时间与吞吐量：Quantum 通常设在 10–100ms；Context Switching Overhead 约 0.1 1 ms，占总时长 1% 左右

优点：对短任务更友好（Fairness ↑）；缺点：长任务 CPU 有更多时间花在 Context Switching 上。

*RR 不一定比 FCFS 更好：如果所有任务长度相同/相近，RR 让所有任务都接近最后时刻才完成；而 FCFS 可以较早完成前几个开始的任务。

*RR 更公平，并不意味着它能最小化 Average Completion Time

Priority and Fairness

Priority Scheduling

Strict Priority Scheduling 始终优先执行最高优先级参数的 READY Job，同一优先级内部使用 RR；高优先级线程 Blocked 时低优先级 Ready 线程可以运行。

Starvation：如果高优先级线程不断有 READY，低优先级线程可能永远不执行

Deadlock（Priority Inversion）：低优先级任务拥有高优先级任务需要的 Mutex Lock，高优先级 BLOCK 后中等优先级任务反复试图获得 Lock 导致卡死。

解决方法：Dynamic priorities 根据任务的 Interactivity、锁竞争、CPU Burst 行为等启发式 Heuristics 调整 base-level priority。

*优化高优先级任务的响应时间，不保证所有任务都公平获得 CPU。

Fairness-ResponseTime Tradeoff

Strict Priority 对低优先级线程不公平（Starvation），提高公平性可能需要牺牲平均响应时间

方案一：给不同优先级队列分配固定 CPU fraction

问题是如果每个优先级的线程数量极端不均会导致“快速通道悖论”：如果大家优先级都高那即使配额再高也不够分——无法适应各队列线程数量及负载变化，导致资源分配不合理。

方案二：提高长期等待任务的优先级（Dynamic Priority 的一种）

UNIX 的部分变体采用这个方案。若系统 overload，所有线程都在等待 CPU，优先级一起升高，最终原本需要快速响应的 Interactive Jobs 受害。

Shortest Job First (SJF) & Shortest Remaining Time First (SRTF)

假设 Scheduler 提前知道每个任务需要多少 CPU 时间，可根据这些信息减少平均完成时间——每次选择 CPU Burst 最短的任务执行 SJF;

平均完成时间 Average Completion Time 明显优化：原因是先执行短任务，完成时间之和更小。

SJF 如果一个短任务在长任务开始执行后才到达必须等后者执行完；SJF 是非抢占式 Non-preemptive 的。

SRTF 是 SJF 的抢占式版本：当一个新的剩余时间更短的任务到达就可以立即抢占。

SRTF 永远比 SJF 好，短任务不会卡在长任务后，average response time 小

若所有任务一样长，SRTF 落回 FCFS 基线（此时 FCFS 就是最佳，RR 最差）

SRTF 特别适合 长-I/O 任务，因为硬件使用率高

如果全是短任务，长任务会被 starve —— unfair

SRTF 可以作为衡量其他 Scheduling Policies 的理论基准，但实际实现受到预测能力的限制，即使 non-malicious 用户/程序也难以预测。

Predicting the Future

*Adaptive Scheduling 自适应调度：假设程序运行行为具有一定的时间相关性，根据线程过去行为预测未来。

比如用 Exponential Averaging 使用过去 CPU Burst 的实际长度，预测下一次 CPU Burst

CSC3150 Lecture11 Scheduling: Fair Sharing, Lottery Scheduling, EEVDF

上节：如何让任务更快完成? FCFS、RR、SJF、SRTF。

本节：如何让多个任务公平地分享 CPU? 如何保证不同任务获得不同份额，同时避免 Starvation?

Multi-Level Feedback Queue (MLFQ)

回顾上节课的 SRTF（Shortest Remaining Time First）：每次选择剩余 CPU Burst 最短的任务执行。

理论上，这有利于降低平均完成时间，但问题是 OS 根本不知道一个程序未来还需要运行多久。所以我们需要一种不依赖未来信息、但又能模仿 SRTF 的调度方法：Multi-Level Feedback Queue（多级反馈队列）。

其核心思想是：根据线程过去的 CPU 使用行为，动态调整其优先级。

例如：A 线程每次只运行 1 ms 就主动阻塞，可能是 I/O-bound；B 线程每次都用完整个 Time Quantum，可能是 CPU-bound。

于是，OS 可以把前者放在较高优先级，把后者逐渐降到较低优先级。

一般的，CPU-Burst 越短，优先级/权重更高。

MLFQ Structure

MLFQ 由多个不同 priority 的队列构成，高优先级队列（foreground）使用 Round Robin 调度，反之 background 使用 FCFS 调度。通常每层 foreground time r quantum 指数级增加。长 CPU 时间任务会被降级到更低优先级队列。假设有三个队列:

最高优先级 Q0：RR Scheduling（Time Quantum = 8 ms）

中等优先级 Q1：RR（q = 16 ms）

最低优先级 Q2：FCFS

一种简朴的 MLFQ 规则:

新 Job 从最高优先级队列开始。

如果 Time Quantum 用完，就降低一个优先级。

如果 Time Quantum 没用完就主动阻塞，可以提高一个优先级（或回到顶部）。

因此 CPU-bound 任务倾向于逐渐降级；I/O-bound 任务倾向于停留在较高优先级。

优先级从高到低 Fixed Priority Scheduling，加上每个队列固定 time slice CPU 时间（越高越长）。就实现在不知道未来 Burst 长度的情况下，利用过去行为近似 SRTF 的调度效果。

MLFQ Defect

假设某个程序本来需要持续计算，但它故意在 Quantum 用完之前发起大量无意义 I/O。

MLFQ 可能被刻意制造的行为误导：如果调度器把短 CPU Burst 和主动阻塞视为交互性强的信号，这个程序就可能维持较高优先级（比如塞 printf）。但如果所有线程都这么做就没有意义了。

*Linux O(1) Scheduler

这是 Linux 历史上的一个调度器，不是当前 Linux 普通任务调度的主要机制。实际应用了 MLFQ 和动态优先级。

① Active Queue 和 Expired Queue:

Active：当前还有可运行时间片的任务。

Expired：用完时间片、等待下一轮的任务。

当 Active 中的任务都处理完后，两个队列交换角色。

运行效果类似于 RR+Priority

② O(1) Scheduling:

通过固定数量（~140）的优先级和 Bitmask 表示优先级满/空，使选取下一个任务的操作不随任务数量线性增长。

任意任务 time out 前，Timeslices/priorities/interactivity credits 等全部计算完毕

IC（Interactivity Credits）：sleep 积攒 IC，CPU-burst 消耗 IC；以及一系列奖励 sleep&I/O 的启发式

Fair Sharing

Fairness 杜绝 Starvation。

*Work-Conserving Scheduler：只要存在可以执行的任务，就不让 CPU 无故 Idle。

例如：A 只需 20% CPU，B 要 100%，假设两者原本平分 50%。A 只用掉 20%，那么剩余 30% 应该给 B，而不是让 CPU 空闲。

Max-Min Fairness

Max-Min Fairness 先让所有任务公平分享资源；若某些任务的需求小于公平份额，满足它，并把剩余资源重新平分给其他任务。 并非无条件让所有任务得到相同资源。

例如总额度 C=100，四个任务需要 [10，20，50，80]，最终分配 [10，20，35，35]。

递归算法（水位上涨 Water Filling）：反复平分剩余资源并检查，如有需求小于配额的，满足它并对剩余任务再次平分剩余资源，循环直到没有小于配额的任务存在。

公平分配公式：sum_{i=1}^{n}min(D_i，f)=C 其中 D 表示各个任务需求，f 表示最终的公平水位。计算时先平分算出 f0 = C/|D|，检查需求-配额，有需要时更新 f 直到

更普遍的，如果有些任务更加重要，根据重要性分配权重 weight。

sum_{i=1}^{n}min(D_i，fω_i)=C 其中 f_i=C{{omega_i}over{sumomega_i}}

Lottery Scheduling

Lottery Scheduling 是一种用随机方式近似 Weighted Fair Sharing 的方法：任务的权重越高，持有的彩票 Tickets 越多，被选中运行的概率越大。Schedule Decision 由概率决定， 也只保证概率意义上的公平。

每个任务持有彩票数为 N，被抽中概率显然是 P_i={{N_i}over{sum_j N_j}}

Lottery Scheduling 的缺点:

某个任务可能连续多次被选中/未选中（不是均匀伪随机）;

短期公平性不稳定，长期平均份额才会趋近于 Tickets 比例。

因此它不能提供严格的短期响应时间保证。

Weighted Fair Queueing、Virtual Time、EEVDF

Weighted Fair Queueing (WFQ)

公平队列最早用于网络带宽分配，后来相同思想也用于 CPU Scheduling。

*Fluid Flow System 流体模型：假设有一条网络链路 C = 10 Mbps，Flow 的需求分别是 r，可以通用 Max-Min Fairness 计算 f，不过多赘述。

流体模型也被称为 Generalized Processor Sharing 广义处理器共享：可以按照 bit 为单位，交替为不同 Flow 提供服务的理想系统。用 Bit-by-Bit Round Robin 模型实现。

（但现实中的 Packet 不一定只有几个 bit，我们不能随意把 Packet 切开，交错发送）

WFQ 模拟 GPS

WFQ 的核心规则：Select packet that finishes first in GPS assuming that there are no future arrivals.

想象每个 Packet 在理想 GPS 中何时完成，优先发送最早完成的 Packet。

例如有两个 Flow，权重相同，各有两个长度为 1 的 Packet。

在 GPS 中，它们交替获得服务；WFQ 可以按照虚拟完成顺序发送。它并不是实际同时传输多个 Packet，而是按照 GPS 的完成顺序安排实际传输。

假设红色 Flow 的权重为 5，另外五个 Flow 的权重分别为 1。所有 Packet 长度都是 1。

在理想 GPS 中，红色 Flow 应获得 50% 资源；但 WFQ 可能使红色 Flow 连续获得 5 个 Packet 的服务，随后其他 Flow 才得到服务。这在长期来看是公平的，但短期内非常不均匀。WFQ 关注理想完成顺序，却不一定保证服务在时间上均匀分布。

Virtual Time 虚拟时间

为什么需要 Virtual Time? 假设三个 Flow 都在发送 Packet，在 GPS 中，三个 Flow 平分带宽。突然第四个 Flow 加入，此时每个 Flow 的带宽份额从 C/3 变成 C/4，于是原先预计的 Packet 完成时间就会改变。

Packet 的实际完成时间可能改变，但它们在 GPS 中的完成先后顺序不会因此改变。

因此，我们不需要一直维护会变化的真实完成时间，而可以维护一个表示服务进度的 Virtual Time。

*System Virtual Time V(t)：描述理想 GPS 系统已经完成了多少轮公平服务

V(t) 斜率表示 backlogged flow 的单位权重获得服务的速度（当有 flow 不再发送信息，斜率增大；加入 flow 斜率减小）

{{dV(t)}over{dt}}={Cover{N(t)}}

Backlogged 指 Flow 仍有等待处理的数据，N(t) 表示当前所有 Backlogged Flows 的权重总和

*Virtual Finish Time：

为什么要用 max：新 Packet 必须同时满足两个条件

它已经到达系统，不能在到达之前开始;

同一 Flow 的前一个 Packet 已经完成，才能轮到它。

EEVDF

Earliest Eligible Virtual Deadline First：优先选择虚拟截止时间最早的可被调度任务

它希望同时实现两个目标:

避免某个 Flow 连续占用资源，导致其他 Flow 长时间等待：Minimize Lag

定义 Lag 为 GPS 和真实系统获得服务量之间的差异，Lag > 0 表示实际服务落后于理想服务

例如上文的红色 Flow 应该获得 50% CPU，WFQ 运行 5ms 后可能 Lag = −2.5ms（实际获得过多服务导致其他 Flow 暂时拿不到资源）

EEVDF 的关键是 Eligibility（可调度资格），即只考虑在理想 GPS 中已经有资格开始接受服务的 Packet。

Eligible 即 Sik ⩽ V(t)：Virtual Starting Time ⩽ Virtual Time

然后在所有 Eligible 的 Packet 中，选择 Virtual Deadline 最早的一个。

CSC3150 Lecture12 Scheduling: Deadlocks

Real-time scheduling 实时调度

*Perdicatability：实时系统的目标不是让任务更快完成，而是可靠地在规定时间内完成。
因此实时系统更关注：Worst-Case Response Time (WCRT)
Hard Real-Time：必须满足 Deadline（超时可能造成严重系统失败），典型调度策略为 Earliest Deadline First (EDF)。
Soft Real-Time：尽可能满足，但允许一定的 Deadline Miss，典型算法如 Constant Bandwidth Server (CBS)

Hard Real-Time：EDF

Round Robin 只是公平轮流分配 CPU Quantum，不考虑哪个任务 Deadline 更近；
实时系统需要每次选择已 ready、尚未完成且绝对 Deadline 最早的任务，即 EDF。它属于 Preemptive Scheduling（可抢占）& Dynamic Priority Scheduling（动态优先级），因为任务优先级不固定，由当前 Job 的绝对 Deadline 决定：新到达的任务 deadline 若更早可打断当前任务。
Periodic Tasks 周期任务：Ti = (Pi, Ci) P 代表周期长，C 为所需 CPU 计算时长。那么第 k+1 次 deadline 为 Dik+1 = Dik + Pi。
EDF Feasibility Test：对 n 个任务，必须满足 sum_{i=1}^{n}{{C_i}over{D_i}}⩽1 才存在调度，否则单核无法完成。
对于周期性任务，利用率 U = sum_{i=1}^{n}{{C_i}over{P_i}}⩽1 表示 CPU Capacity

Multi-core Scheduling

多核调度在算法上和单核相近，但是：
需要 per-core 调度数据结构解决 Cache Coherence
任务究竟该被安排在哪个 CPU Core 上？
*Affinity（处理器亲和性）Scheduling：OS 尽量让 Thread 回到之前执行过的 CPU Core（缓存有可能存有其数据）
Cache 复用（higher hit rate）、Branch Prediction 状态复用、减少不必要的迁移开销；
必要时线程可能迁移：为了 Load Balancing，OS 仍可把任务转移到其他核。
Spinlock 在多核系统中有用——
如果 Lock 已经被占用，线程就不断 Busy Waiting 循环检查，T&S 必须原子操作。
因为持锁线程可能正在另个 Core 执行，且很快就会释放。此时忙等一小段时间可能比进入 Kernel 睡眠再被唤醒更快。但是如果等待时间很长，Spinlock 会浪费 CPU。
T&S 导致多个竞争线程（即使 Lock 一直没有释放）反复执行写操作，可能造成 Cache Line 在不同 CPU Core 之间反复转移，增加 Cache Coherence Traffic（Cache Ping-Pong）。
解决方法是 Test&Test&Set：先用普通 Read 检查 Lock，仅当锁可能空闲才尝试 T&S
T&T&S 避免每次循环都执行昂贵的原子写操作，但锁释放时仍可发生竞争。

Scheduling Evaluation

三种调度算法评价方式：
Deterministic Modeling：给定固定 Workload，直接计算算法表现；
Queueing Models：通过数学模型分析随机 Workload；
Implementation / Simulation：通过实际实现或仿真测试性能。
调度规则和公平性在资源不足时尤其重要；
当 Utilization 接近 100% 时，Response Time 陡增 →∞。
原因是系统接近饱和时，即使新任务数量只增加一点，排队等待时间也可能大幅上升。
因此，合理扩容/硬件提速通常不应该等到资源使用率达到 100% 才进行，在 U-R Load 曲线不再平缓时就应进行。

Priority Inversion

Priority Inversion 是严格优先调度的典型困境：存在三个优先级从低到高的任务，A 首先获得 Lock；C 到达并尝试获取 Lock 并 Block；B 也处于 Ready 状态，Scheduler 会优先 B，它无需锁并可以不断运行，间接阻断 C。这就是 B/C 优先级反转。
解决方法是一种 Dynamic Priority —— Priority Donation/Inheritance（优先级继承）：让持锁的低优先级任务暂时继承等待锁的高优先级任务的优先级，在释放锁后恢复。
Priority Inheritance 不保证彻底消除所有等待，也不能自动解决 Deadlock，它主要解决的是由于中优先级任务抢占而造成的 Priority Inversion 延迟。

Deadlock Condition&Detection

*Starvation 饥饿：thread waits indefinitely，比如优先调度常见低优先级线程长期得不到 CPU 资源。
*Deadlock 死锁：circular waiting for resources 循环资源依赖，A 需要等 B 的资源，且 B 需要等 A。Deadlock 是一种特殊的严重 Starvation（Starvation 不是 Deadlock）。前者可能随调度策略变化或高优先级任务完成而结束（而且不一定是需要结束的负面事件）；后者通常无干预永不结束。
类比单道桥，如过东向车流不断，西向车无法上桥就是 Starvation；一辆西向车趁空档上桥，结果两个方向在桥上堵住就是 Deadlock。

Deadlocks When?

*Locks、Memory、Printers、Terminals、CD drives…（等独占资源）和 Pipes / Sockets（等其他线程）都可能造成 Deadlock。
*Lock Deadlock：
Thread A Thread B ; x.Acquire(); y.Acquire(); ; y.Acquire(); x.Acquire(); ; y.Release(); x.Release(); ; x.Release(); y.Release();
non-deterministic deadlock：这里有概率但不必然发生死锁（更难 debug），取决于线程执行顺序 Interleaving。但不是所有死锁都依赖执行顺序。
若 A B 交替获得 x 和 y，两个线程互相依赖，都无法运行到 Release();
反之某一线程同时获得 x 和 y，就不会死锁。
其他例子比如 Memory Deadlock：有限内存下连续 Allocate 可能引起类似死锁。

Conditions of Deadlocks

死锁可能发生必须同时满足的四个必要条件（Coffman Conditions）：
Mutual Exclusion 互斥：同一个资源在同一时刻只能被一个线程使用。
Hold and Wait 持有并等待：线程已经持有至少一个资源，同时还在等待其他资源。
No Preemption 不可抢占：资源不能被系统强制收回，只能由持有它的线程主动释放。例如不能随意强制解除正在使用的 Mutex。
Circular Wait 循环等待：存在一个线程等待环：T1 等 T2 等…Tn 等 T1，每个线程都在等待下一个线程持有的资源。
Coffman 是 Deadlock 发生的必要但不充分条件，不保证某次执行必然发生死锁。只要设法破坏四个必要条件中的至少一个，就可以防止相应类型的死锁。

Resource Allocation Graph

定义两种节点组成的有向图：
Thread Node：圆形，表示线程 T。
Resource Node：方形，表示资源类型 R；其中的黑点代表资源实例（Instance），其个数 W 表示可独占数
请求边是 Thread 指向 Resource，分配边是 Resource 指向 Thread（以获得）
*图存在 cycle 不代表必然存在 Deadlock。如果环中存在 W > 1 且未全部分配给环内的 T，就不存在死锁。

Deadlock Detection Algorithm

有没有一种顺序，让所有线程都能依次获得所需资源、完成执行并释放资源？
定义三种向量变量，按 R1 到 Rn 维度：
FreeResources 表示当前每种资源还有多少可用实例；
RequestX 表示线程 X 当前还在请求多少资源；
AllocX 表示线程 X 当前已经持有多少资源。
向量之间的比较是逐元素比较，例如 [1,2] ⩽ [2,3] True；[1,3] ⩽ [2,2] False。
Avail = FreeResources ; UNFINISHED = 所有线程 ; repeat: ; done = true ; for each thread X in UNFINISHED: ; if Request[X] <= Avail: # 检查谁当前可完成 ; Avail = Avail + Alloc[X] ; 从 UNFINISHED 移除 X ; done = false ; until done == true
思路：循环检查谁用当前资源可以完成任务并释放持有的资源，最终 UNFINISHED 剩下的线程处于 Deadlock。
Deadlock Detection 使用的是当前 Request，而不是未来可能需要的最大资源 Max。

Dealing with Deadlock

Prevention 预防：设计系统/代码时，破坏死锁必要条件
Avoidance 避免：每次资源分配时，判断是否会进入不安全状态
Detection & Recovery：允许死锁发生，再检测并解决
Ignore（Denial）忽略：不主动检测，交给应用或人工干预

Deadlock Prevention

破坏 Mutual Exclusion
① 资源依赖：提供足够大的资源或无限资源的幻觉（例如 Virtual Mem.）；
② 线程互相依赖（数据依赖）：难以解决，Mutex Lock 保护的共享数据无法简单取消独占。也不可能直接让线程独立（不共享数据）。
破坏 Hold and Wait
① 不允许长期等待：常见于电话公司、以太网，因为需要反复重新尝试所以效率不高；
② All or Nothin' 要求线程开始执行前同时申请所有所需资源：就不会出现多线程分别占有对方需要的资源。问题是资源利用率可能下降，因为线程提前占有资源（即使当前不用），且容易 overestimate 所需资源。
破坏 Circular Wait
① Lock Ordering 强制所有线程按某个全局顺序请求资源：比如全局按 R 序号递增顺序 Acquire()。所有等待关系都只能指向更高编号的资源。若要形成 Cycle 就必须最终从高编号资源等低编号，不可能。
比如 Wormhole-Routed network（四个右转列车在口字岔口堵住）可以用 dimension ordering（先东西后南北）解决

Deadlock Recovery

"破罐破摔"：强制终止 kill 线程并放弃资源
破坏 No Preemption
在不终止线程的前提下，允许 OS 必要时强制收回资源（Preempt 通常是异步 Asynchronous 的）。例如某些 Mem 资源可以被换出（paging out to disk 时线程无法使用）。但不能随意强制释放 Mutex，否则可能破坏数据一致性 Coherency。
让事务回滚（roll-back），释放已占用的资源，常见于数据库系统

Deadlock Avoidance

每次资源分配时，判断是否会进入不安全状态，若是则等待其他现场释放资源，不是就立刻分配资源——
假设系统有若干线程，每个线程可能继续申请资源。一个状态是 Safe 的，当且仅当：
存在至少一种线程完成顺序，使所有线程都能依次获得剩余所需资源、完成任务并释放资源。这个顺序是 Safe Sequence。
反之如果不存在任何 Safe Sequence，未来死锁无法避免就叫 Unsafe State。
*Deadlocked → Unsafe 但 Unsafe 不代表已经 Deadlocked
因为 Unsafe 表示系统无法保证未来所有线程都能完成，并不代表它们现在已经互相等待。

Banker's Algorithm

银行不能仅仅因为现在有钱，就立即批准每一笔贷款。因为批准某笔贷款后，可能出现：每个客户都拿到一部分钱，但没有任何客户能获得足够的钱完成项目。所有客户都无法还款。
银行家算法的目标是：简化忽略利息，在批准资源请求前，模拟分配后的状态，只有仍然存在 Safe Sequence 时才批准。
Max：和 Deadlock Detection Algorithm 不同的是，需要提前知道每个线程最大资源需求。
Alloc：线程已经持有多少资源。Need = Max − Alloc（未来最多继续申请多少资金）
FreeResources：系统目前剩余多少资源。

Deadlock Avoidance Algorithm:

模拟完成时只加 Alloc 而不是 Max。因为线程为了完成任务，可能先临时借用 Need，随后把自己持有的全部资源归还抵消。
运行完成后会得到一条 Safe Sequence 或得到相互 Unsafe 的元素。

CSC3150 Lecture13 现代计算机系统的调度

Firmament (Cluster Scheduling)

以前我们讨论的 CPU Scheduling，通常是多个线程争夺一或多个 CPU，操作系统决定谁先执行
但数据中心有成千上万台服务器，应该把哪个任务分配到哪台机器——Cluster Scheduling 集群调度
*Firmament = Centralized Scheduling + Min-cost Flow + Fast Incremental Optimization
= Good Task Placement + Low Scheduling Latency + Large-scale Cluster Support
Interactive Jobs：交互式任务，要求几秒内完成。
Long-running Services：长期运行的服务，需要稳定性能。
Batch Jobs：批处理任务，通常用于提高资源利用率。
调度器需要同时实现两个目标：
Good Task Placement（好的任务放置）：避免资源冲突，提高整体性能。
Low Scheduling Latency（低调度延迟）：不能让任务长时间等待调度决策。

Centralized vs Distributed Scheduling

全局优化可能很慢：假设有 10,000 个任务和 10,000 台机器。每个任务可能有不同的机器偏好、资源需求和迁移成本。中央调度器必须考虑任务之间的竞争，才能做出整体较好的决定。
而分布式调度器可以更快地作出局部决策，但不一定知道其他机器的实际情况。
Firmament 的目标就是同时获得全局优化能力和低调度延迟。

Min-cost Flow Scheduling

Firmament 将任务分配问题转化为一个 Graph Optimization Problem：一些任务分别一对一指向一些 Machine（现实中可能继续指向其他 Machine）直到任务完成全部指向 Sink。
Min-cost Flow 优化的是整体 Cost，而不是让每个 Task 都选择自己 Cost 最低的 Machine。这也是全局调度和局部贪心调度的区别。
Firmament 不只是调度新任务，也可以重新考虑已经运行的任务：
Migration：把任务从一台机器迁移到另一台机器。
Preemption：暂停或抢占已有任务，为其他任务腾出资源。
因此它可以优化整个 Cluster 的任务分配，而不只是给新任务找一个空位置。

算法调度速度对比

Cost Scaling 大规模集群下过慢
Successive Shortest Path wcc 最快，但实验中扩展到约 100 台机器就很慢
Relaxation wcc 最差，但通常能实现亚秒级调度
Relaxation wcc 虽然很高，但在所研究的任务分配网络中，通常表现最好。原因是它可以利用当前已有的任务分配结果，不必每次从零开始求解。
Relaxation：先保留当前任务分配，再寻找能降低整体 Cost 的调整方式。Incremental Recomputing：新增任务时不必从零计算所有任务的所有分配。可以从当前方案出发，尝试局部调整，寻找新的低成本方案。
Higher Utilization → Potentially Higher Scheduling Cost
当 Cluster Utilization 接近 100% 时，Relaxation 可能变慢。因为机器资源紧张，无法简单地将一个任务移动到空闲位置，可能需要连续调整多个任务。

Gandiva (AI Deep Learning Training Scheduling)

DLT

传统 GPU Scheduler 通常把深度学习训练任务当成普通大数据任务。
Non-preemptive/Exclusive：一旦某任务获得 GPU，就可能一直占有直到完成（类似于 CPU FCFS）
问题一：High Latency——Head-of-line Blocking 队首阻塞（类似于 FCFS Convoy Effect）
较大 DLT 挡住后面小推理任务，需要 time-slicing
问题二：Low Efficiency：初次分配后固定资源分配
导致利用率不均，资源分布不合理，需要 migration
Intra-job predictability：DLT 通常由大量 Mini-batches 构成。同任务的 Mini-batch 执行时间和资源使用模式具有较强的可预测性/周期性。
这意味着调度器可以用 Mini-batch 边界（Iterative boundary）做 time-slicing quantum，而不是随意打断训练——Application Semantics-Aware Scheduling 利用任务本身运行特征调度。

Gandiva

更小 Latency（4.5x）+ 更高 Efficiency（Troughput+26%）的 DLT workflow
*Mechanism 1 — Time-slicing：和传统 CPU Round Robin 很相似，解决长任务阻塞短任务
不同 Job 轮流使用 GPU，而不是一个 Job 长期独占。
Gandiva 的 Time Quantum 约为 q = 1 minute ≳ 100 mini-batches
暂停任务时等待当前 Mini-batch 完成，将必要的 GPU 状态复制到 CPU，暂停当前 Job 并切换到另一个 Job。
在 Mini-batch Boundary 暂停，可以显著降低切换成本（≳ 50x）。因为此时训练任务处于相对明确的阶段边界，不必在一个 Mini-batch 的计算中途保存复杂的临时状态（Application-aware Scheduling）。
*Mechanism 2 — Migration & Packing：
Migration 迁移：解决任务分布不合理、资源碎片化。把一个 Job 从 GPU A 移到 B。整理碎片化资源，使多 GPU 任务获得所需资源；利用不同 GPU 的性能差异；改善整体负载分布。
Packing 打包运行：解决 GPU 资源未充分利用。把多任务放在同 GPU 上运行。多个只使用部分 GPU 资源的任务可能适合同时运行。
GPU Utilization 上升，不一定代表有效吞吐量上升。假设两个任务分别运行时每秒完成 10 个 Mini-batches，Packing 后 GPU Utilization 增加，但由于相互干扰，每个任务每秒只能完成 4 个，那么 Total Throughput 反而下降：4+4=8<10。所以 Packing 并不一定有利。
*Mechanism 3 — Application-aware Profiling：解决无法判断调整是否提高效率
Gandiva 测量吞吐量指标而非靠使用率粗糙地判断：Useful Work = Time per Mini-batch（尝试并测量吞吐量变化确定 Migration/Packing 等调整是否保留）
称为 Introspective Scheduling 自省式调度

Synergy (Multi-resource Scheduling)

Synergy 提出深度学习训练不仅需要 GPU，也需要 CPU 和 Memory，不能只考虑 GPU 数量。
GPU-Proportional Allocation 的问题：传统 GPU-Proportional Allocation 按 GPU 数量比例分配 CPU 和 Memory：R = #R x GPU/#GPU。但不同 DNN 任务对 CPU 和 Memory 的需求差异非常大，不必与 GPU 成比例。
Resource Sensitivity：Synergy 通过分析每个 Job 对 CPU、Memory 等资源的敏感程度，进行更合理的分配。
*Optimistic Profiling：Synergy 使用 Profiling 在专用服务器上测试任务的性能与资源分配之间的关系。改变资源，测量性能变化，找到能够达到最佳性能、同时消耗尽可能少辅助资源的配置。

Job Placement

知道每个任务需要多少资源后，怎样把这些任务合理地放进集群属于：Multi-dimensional Bin Packing 多维（CPU，GPU，RAM...）装箱问题。
任务能否放入某台服务器，需要所有维度都满足 Demand ⩽ AvailableResources 这里的比较是逐分量比较。

Multi-dimensional Bin Packing is NP-hard

这意味着在一般情况下，寻找全局最优方案的计算成本可能非常高。因此 Synergy 使用接近最优的 Heuristic Scheduling。

Synergy 的调度策略

从待运行任务队列中选择 Job。
尝试按 Profiling 得出的理想资源配置放入集群。
如果放不下，尝试使用 GPU-Proportional Share。
必要时调整已运行 Job 的辅助资源配置，释放空间。
重复，直到没有可继续分配的 GPU 或任务。
Synergy 并不是完全抛弃 GPU-Proportional Allocation，而是在资源紧张时把它作为 fallback config。
Makespan（finish time span）改善约 1.4 倍；Average JCT（Job Completion Time）改善约 1.5 倍；99th Percentile JCT 改善约 2 倍。